

SESSION 4

Reasoning & Structured Outputs

Make models reason on purpose — and return output your code can parse.

Mardi 22 septembre · 10h45–12h30 · 1h45

Prompt & Context Engineering — Stefan Duprey



TODAY

Session 4 — plan & goals

You will be able to

- Adopt a branch-per-task Git workflow
- Use CoT, decomposition & self-consistency
- Produce & validate structured JSON output
- Measure whether reasoning actually helps

RUN OF SHOW

0:00	Git — branch per task, good commits
0:15	Reasoning & structured outputs
1:00	Lab — does CoT help? measure it
1:40	Wrap

One branch per task

- Create a branch for each exercise or feature
- Keep main always in a working state
- Commit small and often, in present tense
- A clean history is a gift to your future self



GOOD COMMIT

"Add windowed memory to chat chain" — says what and why. Not "stuff" or "fix".

Why ask a model to reason

Sometimes the fix for a wrong answer is simply giving the model room to think before it commits.

- **Hard tasks fail in one jump**

Multi-step arithmetic, logic and planning collapse when the model tries to answer immediately. Space to work helps.

- **Explicit steps raise accuracy**

Letting the model write intermediate steps before the answer measurably improves results on reasoning-heavy tasks.

- **Steps make errors visible**

When the reasoning is on the page, you can see where it went wrong — and so can a judge. But it costs tokens and latency.



PRINCIPLE

Give the model room to think before it commits — but only turn it on where your eval set shows it actually helps.

Chain-of-thought

The most useful reasoning trick is also the simplest: ask for the working, then the answer.

- **Ask for steps, then the answer**

'Work through it step by step, then give the final answer' is the canonical form. Simple and effective.

- **Delimit the final answer**

Put it after a clear marker so your code can extract it without parsing the reasoning.

- **Modern models often do it unprompted**

Newer models reason by default; you may be steering how much, not whether. Measure to find out.

```
chain-of-thought
```

```
Work through it step by step,  
then give the answer after  
"Answer:"  
on its own line.
```

```
# steps are visible -> checkable
```

Decomposition (least-to-most)

When a problem is too big to solve in one leap, break it into steps you can actually check.

1

Split

Break the problem into an ordered list of smaller sub-questions the model can handle one at a time.

2

Solve in sequence

Answer each sub-question, feeding earlier answers forward as context for the next.

3

Compose

Assemble the sub-answers into the final result. Each step is small enough to get right.

4

Why it works

Small steps are both easier for the model and easier for you to inspect when something goes wrong.

Reason more reliably

A few more tools for when you need the reasoning to be robust, not just present — each with a cost.



Self-consistency

Sample several reasoning paths and take the majority answer. More robust, but multiplies cost — a clear quality/cost trade.



Cost vs benefit

Reasoning burns tokens and time. Skip it for simple lookups; reserve it for tasks that genuinely need it.



Hidden reasoning

Have the model reason internally and output only the conclusion when you don't want to show the working.



Verify

Add a final step that checks the answer against the constraints before returning it. Cheap insurance.

ReAct: reason + act

Reasoning gets more powerful when the model can reach outside itself between thoughts — this is the seed of an agent.

- **Interleave thought and action**

The model alternates Thought → Action → Observation, using tools between reasoning steps.

- **Actions call tools**

Search, a calculator, code execution — the model decides what to call and reads the result back.

- **This loop becomes an agent**

ReAct is the seed of the tool-using agents we build in S6. Same idea, more structure.

ReAct trace

```
Thought: need today's FX rate
Action: search("EUR USD")
Observation: 1.09
Thought: now I can answer
Answer: about 1.09
```


Structured output & repair

This is the piece that makes an LLM safe to wire into real software: output your code can actually trust.

- **Specify keys, types, allowed values**

Tell the model the exact JSON you expect. Use the provider's JSON/schema mode where available.

- **Parse → validate → repair**

Parse the output, validate it against a schema, and on failure send the error back and ask for a corrected object.

- **Never regex prose**

Don't scrape structure out of free text. Make the model emit structure you can load directly.

```
schema + repair
```

```
Return ONLY JSON: { label: str,  
  confidence: 0.0-1.0 }
```

```
# on parse failure:  
send the error back, ask for a  
corrected object, retry once
```

Lab — Does reasoning actually help?

LOCAL · OLLAMA

`prompt-engineering-course/04_reasoning_structure`

TASKS

1. Branch from your eval harness (S3)
2. Run one task with and without chain-of-thought
3. Score both on the same eval set
4. Decide from the numbers, then commit the winner with a one-line conclusion



DELIVERABLE

A measured comparison of CoT vs no-CoT on your eval set, with the token-cost trade noted in the commit message.

Recap — Session 4



Explicit reasoning trades tokens for accuracy



Self-consistency and a verify step add reliability



Structured output = schema + validate + repair



Let the eval set decide whether reasoning helps

NEXT SESSION

Context engineering I

The reframe: from wording to context. Retrieval as prompting, grounding, citation, and context rot.